

from y . At time $y.f$, therefore, all vertices in C' are still white. Thus, for any vertex $w \in C'$, we have $w.f > y.f$, which implies that $f(C') > f(C)$. ■

Corollary 20.15

Let C and C' be distinct strongly connected components in directed graph $G = (V, E)$, and suppose that $f(C) > f(C')$. Then E^T contains no edge (v, u) such that $u \in C'$ and $v \in C$.

Proof The contrapositive of Lemma 20.14 says that if $f(C') < f(C)$, then there is no edge $(u, v) \in E$ such that $u \in C'$ and $v \in C$. Because the strongly connected components of G and G^T are the same, if there is no such edge $(u, v) \in E$, then there is no edge $(v, u) \in E^T$ such that $u \in C'$ and $v \in C$. ■

Corollary 20.15 provides the key to understanding why the strongly connected components algorithm works. Let's examine what happens during the second depth-first search, which is on G^T . The search starts from the vertex x whose finish time from the first depth-first search is maximum. This vertex belongs to some strongly connected component C , and since $x.f$ is maximum, $f(C)$ is maximum over all strongly connected components. When the search starts from x , it visits all vertices in C . By Corollary 20.15, G^T contains no edges from C to any other strongly connected component, and so the search from x never visits vertices in any other component. Thus, the tree rooted at x contains exactly the vertices of C . Having completed visiting all vertices in C , the second depth-first search selects as a new root a vertex from some other strongly connected component C' whose finish time $f(C')$ is maximum over all components other than C . Again, the search visits all vertices in C' . But by Corollary 20.15, if any edges in G^T go from C' to any other component, they must go to C , which the second depth-first search has already visited. In general, when the depth-first search of G^T in line 3 visits any strongly connected component, any edges out of that component must be to components that the search has already visited. Each depth-first tree, therefore, corresponds to exactly one strongly connected component. The following theorem formalizes this argument.

Theorem 20.16

The STRONGLY-CONNECTED-COMPONENTS procedure correctly computes the strongly connected components of the directed graph G provided as its input.

Proof We argue by induction on the number of depth-first trees found in the depth-first search of G^T in line 3 that the vertices of each tree form a strongly connected component. The inductive hypothesis is that the first k trees produced

in line 3 are strongly connected components. The basis for the induction, when $k = 0$, is trivial.

In the inductive step, we assume that each of the first k depth-first trees produced in line 3 is a strongly connected component, and we consider the $(k + 1)$ st tree produced. Let the root of this tree be vertex u , and let u be in strongly connected component C . Because of how the depth-first search chooses roots in line 3, $u.f = f(C) > f(C')$ for any strongly connected component C' other than C that has yet to be visited. By the inductive hypothesis, at the time that the search visits u , all other vertices of C are white. By the white-path theorem, therefore, all other vertices of C are descendants of u in its depth-first tree. Moreover, by the inductive hypothesis and by Corollary 20.15, any edges in G^T that leave C must be to strongly connected components that have already been visited. Thus, no vertex in any strongly connected component other than C is a descendant of u during the depth-first search of G^T . The vertices of the depth-first tree in G^T that is rooted at u form exactly one strongly connected component, which completes the inductive step and the proof. ■

Here is another way to look at how the second depth-first search operates. Consider the component graph $(G^T)^{\text{SCC}}$ of G^T . If you map each strongly connected component visited in the second depth-first search to a vertex of $(G^T)^{\text{SCC}}$, the second depth-first search visits vertices of $(G^T)^{\text{SCC}}$ in the reverse of a topologically sorted order. If you reverse the edges of $(G^T)^{\text{SCC}}$, you get the graph $((G^T)^{\text{SCC}})^T$. Because $((G^T)^{\text{SCC}})^T = G^{\text{SCC}}$ (see Exercise 20.5-4), the second depth-first search visits the vertices of G^{SCC} in topologically sorted order.

Exercises

20.5-1

How can the number of strongly connected components of a graph change if a new edge is added?

20.5-2

Show how the procedure STRONGLY-CONNECTED-COMPONENTS works on the graph of Figure 20.6. Specifically, show the finish times computed in line 1 and the forest produced in line 3. Assume that the loop of lines 5–7 of DFS considers vertices in alphabetical order and that the adjacency lists are in alphabetical order.

20.5-3

Professor Bacon rewrites the algorithm for strongly connected components to use the original (instead of the transpose) graph in the second depth-first search and

scan the vertices in order of *increasing* finish times. Does this modified algorithm always produce correct results?

20.5-4

Prove that for any directed graph G , the transpose of the component graph of G^T is the same as the component graph of G . That is, $((G^T)^{\text{SCC}})^T = G^{\text{SCC}}$.

20.5-5

Give an $O(V + E)$ -time algorithm to compute the component graph of a directed graph $G = (V, E)$. Make sure that there is at most one edge between two vertices in the component graph your algorithm produces.

20.5-6

Give an $O(V + E)$ -time algorithm that, given a directed graph $G = (V, E)$, constructs another graph $G' = (V, E')$ such that G and G' have the same strongly connected components, G' has the same component graph as G , and $|E'|$ is as small as possible.

20.5-7

A directed graph $G = (V, E)$ is *semiconnected* if, for all pairs of vertices $u, v \in V$, we have $u \rightsquigarrow v$ or $v \rightsquigarrow u$. Give an efficient algorithm to determine whether G is semiconnected. Prove that your algorithm is correct, and analyze its running time.

20.5-8

Let $G = (V, E)$ be a directed graph, and let $l : V \rightarrow \mathbb{R}$ be a function that assigns a real-valued label l to each vertex. For vertices $s, t \in V$, define

$$\Delta l(s, t) = \begin{cases} l(t) - l(s) & \text{if there is a path from } s \text{ to } t \text{ in } G, \\ -\infty & \text{otherwise.} \end{cases}$$

Give an $O(V + E)$ -time algorithm to find vertices s and t such that $\Delta l(s, t)$ is maximum over all pairs of vertices. (*Hint:* Use Exercise 20.5-5.)

Problems

20-1 Classifying edges by breadth-first search

A depth-first forest classifies the edges of a graph into tree, back, forward, and cross edges. A breadth-first tree can also be used to classify the edges reachable from the source of the search into the same four categories.

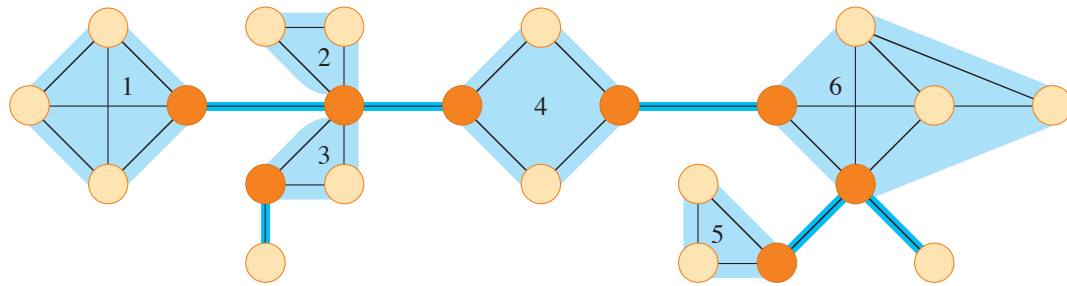


Figure 20.10 The articulation points, bridges, and biconnected components of a connected, undirected graph for use in Problem 20-2. The articulation points are the orange vertices, the bridges are the dark blue edges, and the biconnected components are the edges in the light blue regions, with a *bcc* numbering shown.

- a.** Prove that in a breadth-first search of an undirected graph, the following properties hold:
1. There are no back edges and no forward edges.
 2. If (u, v) is a tree edge, then $v.d = u.d + 1$.
 3. If (u, v) is a cross edge, then $v.d = u.d$ or $v.d = u.d + 1$.
- b.** Prove that in a breadth-first search of a directed graph, the following properties hold:
1. There are no forward edges.
 2. If (u, v) is a tree edge, then $v.d = u.d + 1$.
 3. If (u, v) is a cross edge, then $v.d \leq u.d + 1$.
 4. If (u, v) is a back edge, then $0 \leq v.d \leq u.d$.

20-2 Articulation points, bridges, and biconnected components

Let $G = (V, E)$ be a connected, undirected graph. An **articulation point** of G is a vertex whose removal disconnects G . A **bridge** of G is an edge whose removal disconnects G . A **biconnected component** of G is a maximal set of edges such that any two edges in the set lie on a common simple cycle. Figure 20.10 illustrates these definitions. You can determine articulation points, bridges, and biconnected components using depth-first search. Let $G_\pi = (V, E_\pi)$ be a depth-first tree of G .

- a.** Prove that the root of G_π is an articulation point of G if and only if it has at least two children in G_π .

b. Let v be a nonroot vertex of G_π . Prove that v is an articulation point of G if and only if v has a child s such that there is no back edge from s or any descendant of s to a proper ancestor of v .

c. Let

$$v.\text{low} = \min \begin{cases} v.d, \\ w.d : (u, w) \text{ is a back edge for some descendant } u \text{ of } v. \end{cases}$$

Show how to compute $v.\text{low}$ for all vertices $v \in V$ in $O(E)$ time.

d. Show how to compute all articulation points in $O(E)$ time.

e. Prove that an edge of G is a bridge if and only if it does not lie on any simple cycle of G .

f. Show how to compute all the bridges of G in $O(E)$ time.

g. Prove that the biconnected components of G partition the nonbridge edges of G .

h. Give an $O(E)$ -time algorithm to label each edge e of G with a positive integer $e.\text{bcc}$ such that $e.\text{bcc} = e'.\text{bcc}$ if and only if e and e' belong to the same biconnected component.

20-3 Euler tour

An **Euler tour** of a strongly connected, directed graph $G = (V, E)$ is a cycle that traverses each edge of G exactly once, although it may visit a vertex more than once.

a. Show that G has an Euler tour if and only if $\text{in-degree}(v) = \text{out-degree}(v)$ for each vertex $v \in V$.

b. Describe an $O(E)$ -time algorithm to find an Euler tour of G if one exists. (*Hint:* Merge edge-disjoint cycles.)

20-4 Reachability

Let $G = (V, E)$ be a directed graph in which each vertex $u \in V$ is labeled with a unique integer $L(u)$ from the set $\{1, 2, \dots, |V|\}$. For each vertex $u \in V$, let $R(u) = \{v \in V : u \rightsquigarrow v\}$ be the set of vertices that are reachable from u . Define $\min(u)$ to be the vertex in $R(u)$ whose label is minimum, that is, $\min(u)$ is the vertex v such that $L(v) = \min\{L(w) : w \in R(u)\}$. Give an $O(V + E)$ -time algorithm that computes $\min(u)$ for all vertices $u \in V$.

20-5 Inserting and querying vertices in planar graphs

A *planar* graph is an undirected graph that can be drawn in the plane with no edges crossing. Euler proved that every planar graph has $|E| < 3|V|$.

Consider the following two operations on a planar graph G :

- $\text{INSERT}(G, v, \text{neighbors})$ inserts a new vertex v into G , where *neighbors* is an array (possibly empty) of vertices that have already been inserted into G and will become all the neighbors of v in G when v is inserted.
- $\text{NEWEST-NEIGHBOR}(G, v)$ returns the neighbor of vertex v that was most recently inserted into G , or NIL if v has no neighbors.

Design a data structure that supports these two operations such that NEWEST-NEIGHBOR takes $O(1)$ worst-case time and INSERT takes $O(1)$ amortized time. Note that the length of the array *neighbors* given to INSERT may vary. (*Hint: Use a potential function for the amortized analysis.*)

Chapter notes

Even [137] and Tarjan [429] are excellent references for graph algorithms.

Breadth-first search was discovered by Moore [334] in the context of finding paths through mazes. Lee [280] independently discovered the same algorithm in the context of routing wires on circuit boards.

Hopcroft and Tarjan [226] advocated the use of the adjacency-list representation over the adjacency-matrix representation for sparse graphs and were the first to recognize the algorithmic importance of depth-first search. Depth-first search has been widely used since the late 1950s, especially in artificial intelligence programs.

Tarjan [426] gave a linear-time algorithm for finding strongly connected components. The algorithm for strongly connected components in Section 20.5 is adapted from Aho, Hopcroft, and Ullman [6], who credit it to S. R. Kosaraju (unpublished) and Sharir [408]. Dijkstra [117, Chapter 25] also developed an algorithm for strongly connected components that is based on contracting cycles. Subsequently, Gabow [163] rediscovered this algorithm. Knuth [259] was the first to give a linear-time algorithm for topological sorting.

21 Minimum Spanning Trees

Electronic circuit designs often need to make the pins of several components electrically equivalent by wiring them together. To interconnect a set of n pins, the designer can use an arrangement of $n - 1$ wires, each connecting two pins. Of all such arrangements, the one that uses the least amount of wire is usually the most desirable.

To model this wiring problem, use a connected, undirected graph $G = (V, E)$, where V is the set of pins, E is the set of possible interconnections between pairs of pins, and for each edge $(u, v) \in E$, a weight $w(u, v)$ specifies the cost (amount of wire needed) to connect u and v . The goal is to find an acyclic subset $T \subseteq E$ that connects all of the vertices and whose total weight

$$w(T) = \sum_{(u,v) \in T} w(u, v)$$

is minimized. Since T is acyclic and connects all of the vertices, it must form a tree, which we call a **spanning tree** since it “spans” the graph G . We call the problem of determining the tree T the **minimum-spanning-tree problem**.¹ Figure 21.1 shows an example of a connected graph and a minimum spanning tree.

This chapter studies two ways to solve the minimum-spanning-tree problem. Kruskal’s algorithm and Prim’s algorithm both run in $O(E \lg V)$ time. Prim’s algorithm achieves this bound by using a binary heap as a priority queue. By using Fibonacci heaps instead (see page 478), Prim’s algorithm runs in $O(E + V \lg V)$ time. This bound is better than $O(E \lg V)$ whenever $|E|$ grows asymptotically faster than $|V|$.

¹ The phrase “minimum spanning tree” is a shortened form of the phrase “minimum-weight spanning tree.” There is no point in minimizing the number of edges in T , since all spanning trees have exactly $|V| - 1$ edges by Theorem B.2 on page 1169.

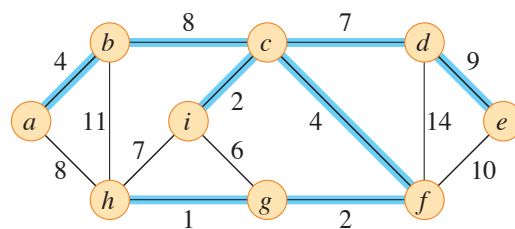


Figure 21.1 A minimum spanning tree for a connected graph. The weights on edges are shown, and the blue edges form a minimum spanning tree. The total weight of the tree shown is 37. This minimum spanning tree is not unique: removing the edge (b, c) and replacing it with the edge (a, h) yields another spanning tree with weight 37.

The two algorithms are greedy algorithms, as described in Chapter 15. Each step of a greedy algorithm must make one of several possible choices. The greedy strategy advocates making the choice that is the best at the moment. Such a strategy does not generally guarantee that it always finds globally optimal solutions to problems. For the minimum-spanning-tree problem, however, we can prove that certain greedy strategies do yield a spanning tree with minimum weight. Although you can read this chapter independently of Chapter 15, the greedy methods presented here are a classic application of the theoretical notions introduced there.

Section 21.1 introduces a “generic” minimum-spanning-tree method that grows a spanning tree by adding one edge at a time. Section 21.2 gives two algorithms that implement the generic method. The first algorithm, due to Kruskal, is similar to the connected-components algorithm from Section 19.1. The second, due to Prim, resembles Dijkstra’s shortest-paths algorithm (Section 22.3).

Because a tree is a type of graph, in order to be precise we must define a tree in terms of not just its edges, but its vertices as well. Because this chapter focuses on trees in terms of their edges, we’ll implicitly understand that the vertices of a tree T are those that some edge of T is incident on.

21.1 Growing a minimum spanning tree

The input to the minimum-spanning-tree problem is a connected, undirected graph $G = (V, E)$ with a weight function $w : E \rightarrow \mathbb{R}$. The goal is to find a minimum spanning tree for G . The two algorithms considered in this chapter use a greedy approach to the problem, although they differ in how they apply this approach.

This greedy strategy is captured by the procedure `GENERIC-MST` on the facing page, which grows the minimum spanning tree one edge at a time. The generic method manages a set A of edges, maintaining the following loop invariant:

Prior to each iteration, A is a subset of some minimum spanning tree.

GENERIC-MST(G, w)

```

1   $A = \emptyset$ 
2  while  $A$  does not form a spanning tree
3      find an edge  $(u, v)$  that is safe for  $A$ 
4       $A = A \cup \{(u, v)\}$ 
5  return  $A$ 

```

Each step determines an edge (u, v) that the procedure can add to A without violating this invariant, in the sense that $A \cup \{(u, v)\}$ is also a subset of a minimum spanning tree. We call such an edge a *safe edge* for A , since it can be added safely to A while maintaining the invariant.

This generic algorithm uses the loop invariant as follows:

Initialization: After line 1, the set A trivially satisfies the loop invariant.

Maintenance: The loop in lines 2–4 maintains the invariant by adding only safe edges.

Termination: All edges added to A belong to a minimum spanning tree, and the loop must terminate by the time it has considered all edges. Therefore, the set A returned in line 5 must be a minimum spanning tree.

The tricky part is, of course, finding a safe edge in line 3. One must exist, since when line 3 is executed, the invariant dictates that there is a spanning tree T such that $A \subseteq T$. Within the **while** loop body, A must be a proper subset of T , and therefore there must be an edge $(u, v) \in T$ such that $(u, v) \notin A$ and (u, v) is safe for A .

The remainder of this section provides a rule (Theorem 21.1) for recognizing safe edges. The next section describes two algorithms that use this rule to find safe edges efficiently.

We first need some definitions. A *cut* $(S, V - S)$ of an undirected graph $G = (V, E)$ is a partition of V . Figure 21.2 illustrates this notion. We say that an edge $(u, v) \in E$ *crosses* the cut $(S, V - S)$ if one of its endpoints belongs to S and the other belongs to $V - S$. A cut *respects* a set A of edges if no edge in A crosses the cut. An edge is a *light edge* crossing a cut if its weight is the minimum of any edge crossing the cut. There can be more than one light edge crossing a cut in the case of ties. More generally, we say that an edge is a *light edge* satisfying a given property if its weight is the minimum of any edge satisfying the property.

The following theorem gives the rule for recognizing safe edges.

Theorem 21.1

Let $G = (V, E)$ be a connected, undirected graph with a real-valued weight function w defined on E . Let A be a subset of E that is included in some minimum

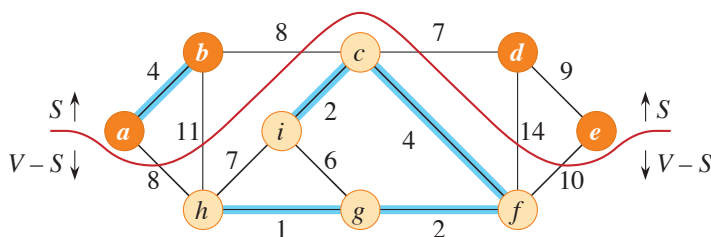


Figure 21.2 A cut $(S, V - S)$ of the graph from Figure 21.1. Orange vertices belong to the set S , and tan vertices belong to $V - S$. The edges crossing the cut are those connecting tan vertices with orange vertices. The edge (d, c) is the unique light edge crossing the cut. Blue edges form a subset A of the edges. The cut $(S, V - S)$ respects A , since no edge of A crosses the cut.

spanning tree for G , let $(S, V - S)$ be any cut of G that respects A , and let (u, v) be a light edge crossing $(S, V - S)$. Then, edge (u, v) is safe for A .

Proof Let T be a minimum spanning tree that includes A , and assume that T does not contain the light edge (u, v) , since if it does, we are done. We'll construct another minimum spanning tree T' that includes $A \cup \{(u, v)\}$ by using a cut-and-paste technique, thereby showing that (u, v) is a safe edge for A .

The edge (u, v) forms a cycle with the edges on the simple path p from u to v in T , as Figure 21.3 illustrates. Since u and v are on opposite sides of the cut $(S, V - S)$, at least one edge in T lies on the simple path p and also crosses the cut. Let (x, y) be any such edge. The edge (x, y) is not in A , because the cut respects A . Since (x, y) is on the unique simple path from u to v in T , removing (x, y) breaks T into two components. Adding (u, v) reconnects them to form a new spanning tree $T' = (T - \{(x, y)\}) \cup \{(u, v)\}$.

We next show that T' is a minimum spanning tree. Since (u, v) is a light edge crossing $(S, V - S)$ and (x, y) also crosses this cut, $w(u, v) \leq w(x, y)$. Therefore,

$$\begin{aligned} w(T') &= w(T) - w(x, y) + w(u, v) \\ &\leq w(T). \end{aligned}$$

But T is a minimum spanning tree, so that $w(T) \leq w(T')$, and thus, T' must be a minimum spanning tree as well.

It remains to show that (u, v) is actually a safe edge for A . We have $A \subseteq T'$, since $A \subseteq T$ and $(x, y) \notin A$, and thus, $A \cup \{(u, v)\} \subseteq T'$. Consequently, since T' is a minimum spanning tree, (u, v) is safe for A . ■

Theorem 21.1 provides insight into how the GENERIC-MST method works on a connected graph $G = (V, E)$. As the method proceeds, the set A is always acyclic, since it is a subset of a minimum spanning tree and a tree may not contain a cycle.